July 2, 2017

Cadabra II

A field-theory motivated approach to symbolic computer algebra

Tutorial and reference guide

Cadabra is a computer algebra system for the manipulation of tensorial mathematical expressions such as they occur in "field theory problems". It is aimed at, but not necessarily restricted to, high-energy physicists. It is constructed as a simple tree-manipulating core, a large collection of standalone algorithmic modules which act on the expression tree, and a set of modules responsible for output of nodes in the tree. All of these parts are written in C++. The input and output formats closely follow TEX, which in many cases means that cadabra is much simpler to use than other similar programs. It intentionally does not contain its own programming language; instead, new functionality is added by writing new modules in C++.

This document contains a description of the core program as well as a detailed listing of the functionality of the currently available algorithm modules. Given the origin of cadabra, the bias is currently towards algorithms for problems in (quantum) field theory, general relativity, group theory and related areas. The last part of this text consists of a user guide on how to implement new algorithmic and display modules and thereby extend the functionality of cadabra.

The software is available for download under the terms of the GNU General Public License from http://cadabra.phi-sci.com/ .

A paper describing the design of Cadabra II is available as

> "*Introducing Cadabra: a symbolic computer algebra system for field theory problems*",
> Kasper Peeters, also available as preprint `hep-th/0701238`.

If you use Cadabra II in your own work, please cite that paper.

http://maths.dur.ac.uk/users/kasper.peeters/

mailto:kasper.peeters@gmail.com

Table of Contents

Overview and motivation

> "*I think that several of you are missing the obvious. The majority of people use these programs as symbolic calculators – occasionally. As such they want their input and output to match what they would have written with pencil and paper.*" `soft-sys.math.maple`, 2002

Cadabra is a computer algebra system for the manipulation of what could loosely be called *tensorial expressions*. It is aimed at, but not necessarily restricted to, theoretical high-energy physicists. The program's interface, storage system and underlying philosophy differ substantially from other computer algebra systems. Its main characteristics are:

- Usage of TEX notation for both input and output, which eliminates many errors in transcribing problems from paper to computer and back.
- Built-in understanding of dummy indices and dummy symbols, including their automatic relabelling when necessary. Powerful algorithms for canonicalisation of objects with index symmetries, both mono-term and multi-term.
- A new way to deal with products of non-commuting objects, enabling a notation which is identical to standard physicist's notation (i.e. no need for special non-commuting product operators).
- A flexible way to associate meaning ("type information") to tensors by attaching them to "properties".
- An optional unlimited undo system. Interactive calculations can be undone to arbitrary level without requiring a full re-evaluation of the entire calculation.
- A simple and documented way to add new algorithms in the form of C++ modules, which directly operate on the internal expression tree.

This document contains a small tutorial, an extensive reference guide and a technical summary of the program's internals together with a guide on how to write new algorithm modules in C++.

Acknowledgements

This program uses code written by several other people. The tensor monomial canonicalisation routines rely on the `xPerm` code written by José Martin-Garcia [**?**]). All representation-theory related problems are handled by the `LiE` software by Marc van Leeuwen, Arjeh Cohen and Bert Lisser [**?**].

The name Cadabra is an implicit acknowledgement to Mees de Roo, who introduced me to his (so far unpublished) Pascal program `Abra` in the fall of 2000. This program has an extremely physicist-friendly way of dealing with fermions and tensor symmetries, and a formula history mechanism still not found in any other comparable computer algebra system. Cadabra was originally planned to be "my private C++ version of `Abra`", and even though it does not show much similarity anymore, the development was to a large extent inspired by `Abra`.

Tutorial

The best way to explain the advantages of a new computer algebra system is to demonstrate its ease-of-use for a real-world problem. But if you lack the patience to even read the tutorial on the next few pages, at least read the remainder of this page for an absolute minimum of information. All input in the examples is prefixed with a "▷" symbol.

- Start the program by typing "`prompt cadabra`". Quit with the "`@quit`" command.
- Tensor expressions are entered as in TEX, with subscripts and superscripts as you know them, e.g.

```
▷ A_{m n} B^{m q} R^{n}_{q};
```

  Input lines are terminated with ";", ":" or "." punctuation; see section **??** on page **??** for information on what these characters mean.

- Tensors carry properties, such as "being a Riemann tensor". These properties are attached by using a double-double dot notation,

```
▷ R_{m n p q}::RiemannTensor.
▷ g_{m n}::Metric.
▷ \psi::Spinor.
```

  A list of all properties and the place where they are described in this manual can be found on page **??**.

- For many operations cadabra needs to know about the names which it can use for 'dummy' indices. You declare dummy indices as

```
▷ {m,n,p,q}::Indices(vector).
```

  where "`vector`" is a chosen name for this index set. See section **??** on page **??**.

- For many other operators, cadabra needs to know the range over which indices run. Set these index ranges by attaching the `Integer` property to the indices, e.g.

```
▷ {m,n,p,q}::Integer(0..10).
```

- Expressions can be given a label so you can refer to them again later; you do this by writing the label before the expression and a ":=" in between,

```
▷ MaxwellEom:= \diff{F^{m n}}_{n} = 0;
```

- All things starting with '@' are commands (also called "active nodes"). Some of the frequently used commands are `@substitute` (page **??**), `@canonicalise` (page **??**) and `@collect_terms` (page **??**).

2.1 Web tutorials and sample notebooks

Apart from the tutorials listed below, there is a growing collection of sample notebooks available on the cadabra web site. In addition, help is available through the mailing list.

2.2 Tutorial 1: Tensor monomials and multi-term symmetries

Cadabra contains powerful algorithms to bring any tensorial expression into a canonical form. For multi-term symmetries, cadabra relies on Young tableau methods to generate a canonical form for tensor monomials. [1]

As an example, consider the identity

$$W_{pqrs}W_{ptru}W_{tvqw}W_{uvsw} - W_{pqrs}W_{pqtu}W_{rvtw}W_{svuw} = W_{mnab}W_{npbc}W_{mscd}W_{spda} - \frac{1}{4}W_{mnab}W_{psba}W_{mpcd}W_{nsdc}\,. \quad (2.1)$$

in which W_{mnpq} is a Weyl tensor (all contracted indices have been written as subscripts for easier readability). Proving this identity requires multiple uses of the Ricci cyclic identity,

$$W_{m[npq]} = 0\,. \quad (2.2)$$

With cadabra's Young tableau methods the proof is simple. We first declare our objects and input the identity which we want to prove,

```
▷ {m,n,p,q,r,s,t,u,v,w,a,b,c,d,e,f}::Indices(vector).
▷ W_{m n p q}::WeylTensor.

▷ W_{p q r s} W_{p t r u} W_{t v q w} W_{u v s w}
    - W_{p q r s} W_{p q t u} W_{r v t w} W_{s v u w}
    - W_{m n a b} W_{n p b c} W_{m s c d} W_{s p d a}
    + (1/4) W_{m n a b} W_{p s b a} W_{m p c d} W_{n s d c};
```

Using a Young projector to project all Weyl tensors onto a form which shows the Ricci symmetry in manifest form is done with

```
▷ @young_project_tensor!(%){ModuloMonoterm};
```

This algorithm knows that the Weyl tensor sits in the $\tilde{\square\!\square}$ representation of the rotation group SO(d), and effectively leads to a replacement

$$W_{mnpq} \to \frac{2}{3}W_{mnpq} - \frac{1}{3}W_{mqnp} + \frac{1}{3}W_{mpnq}\,. \quad (2.3)$$

We then expand the products of sums and canonicalise using mono-term symmetries,

[1] The user interface for multi-term symmetries is under active development and will simplify substantially in upcoming releases.

```
@distribute!(%):
@canonicalise!(%):
@rename_dummies!(%):
@collect_terms!(%);
```

The last line produces the expected "zero". A slightly more complicated multi-term example can be found in TEXmacs format in `texmacs/showcase1.tm`.

2.3 Tutorial 2: Tensor monomials, part two

It is easy to generate complete bases of tensor monomials, for any arbitrary tensors (not necessarily for tensors in irreps, as in the previous example). As an example, let us show how this works for a basis constructed from three powers of the Riemann tensor. All that is required is

```
{m,n,p,q,r,s,t,u,v,w,a,b}::Indices(vector).
{m,n,p,q,r,s,t,u,v,w,a,b}::Integer(0..9).

R_{m n p q}::RiemannTensor.

basisR3:= R_{m n p q} R_{r s t u} R_{v w a b};
@all_contractions(%);
```

The result can be further simplified using

```
@canonicalise!(%):
@substitute!(%)( R_{m n m n} -> R ):
@substitute!(%)( R_{m n m p} -> R_{n p} ):
@substitute!(%)( R_{n m p m} -> R_{n p} ):
@substitute!(%)( R_{n m m p} -> R_{n p} );
```

which leads something like[2]

```
basisR3:= \{ R_{m n p q} * R_{m p r s} * R_{n r q s},
             R * R_{q r} * R_{q r},
             R_{n p} * R_{n q p r} * R_{q r},
             R_{n p} * R_{n q r s} * R_{p r q s},
             R * R_{p q r s} * R_{p q r s},
             R_{n p} * R_{n r} * R_{p r},
             R_{m n p q} * R_{m r p s} * R_{n r q s},
             R * R * R \};
```

This result is equivalent to the basis given in the "$\mathcal{R}^0_{6,3}$" table on page 1184 of [**?**].

[2]The algorithm involves a random step which implies that the basis is not always the same, though it is always complete. Further improvements are in preparation which will eliminate this randomness (and also lead to a speedup).

2.4 Tutorial 3: World-sheet supersymmetry

Cadabra not only deals with bosonic tensors, but also with fermionic objects, i.e. anti-commuting variables. A simple example on which to illustrate their use is the Ramond-Neveu-Schwarz superstring action. We will here show how one uses cadabra to show invariance of this action under supersymmetry (a calculation which is easy to do by hand, but illustrates several aspects of cadabra nicely). We will use a conformal gauge and complex coordinates.

We first define the properties of all the symbols which we will use,

```
▷ {\del{#}, \delbar{#}}::Derivative.
▷ {\Psi_\mu, \Psibar_\mu, \eps, \epsbar}::AntiCommuting.
▷ {\Psi_\mu, \Psibar_\mu, \eps, \epsbar}::SelfAntiCommuting.
▷ {\Psi_\mu, \Psibar_\mu, X_\mu}::Depends(\del,\delbar).
▷ {\Psi_\mu, \Psibar_\mu, \eps, \epsbar, X_\mu, i}::SortOrder.
```

All objects are by default commuting, so the bosons do not have to be declared separately. You can at any time get a list of the declared properties by using the command `@proplist`.

If you try this example in the graphical front-end, `\\del`, `\\eps` and so on would not print correctly as these are not valid LaTeX symbols. However, you can tell cadabra to print such symbols by using the `LaTeXForm` property,

```
▷ \del{#}::LaTeXForm("\partial").
▷ \delbar{#}::LaTeXForm("\bar{\partial}").
▷ \eps::LaTeXForm("\epsilon").
▷ \epsbar::LaTeXForm("\bar{\epsilon}").
▷ \Psibar{#}::LaTeXForm("\bar{\Psi}").
```

If you use the command-line version, these `LaTeXForm` properties are not needed.

Now we have to input the action density (see [**?**] for the conventions used here)

```
▷ action:= \del{X_\mu} \delbar{X_\mu}
         + i \Psi_\mu \delbar{\Psi_\mu} + i \Psibar_\mu \del{\Psibar_\mu};
```

Observe how we wrapped the `\del` and `\delbar` operators around the objects on which they are supposed to act. We are now ready to perform the supersymmetry transformation. This is done by using the `@vary` algorithm,

```
▷ @vary(%)( X_\mu       ->   i \epsbar \Psi_\mu + i \eps \Psibar_\mu,
            \Psi_\mu    -> - \epsbar \del{X_\mu},
            \Psibar_\mu -> - \eps \delbar{X_\mu} );

▷ @distribute!(%);
▷ @prodrule!(%);
```

The `@prodrule` command has applied the Leibnitz rule on the derivatives, so that the derivative of a product becomes a sum of terms. We expand again the products of sums, and use `@unwrap` to take everything out of the derivatives which does not depend on it,

```
▷ @distribute!(%);
▷ @unwrap!(%);
▷ @sort_product!(%);
```

At this stage we are left with an expression which still contains double derivatives. In order to write this in a canonical form, we eliminate all double derivatives by doing one partial integration. This is done by first marking the derivatives which we want to partially integrate, and then using `@pintegrate`,

```
▷ @substitute!(%)( \del{\delbar{X_{\mu}}} -> \pdelbar{\del{X_{\mu}}} ):
▷ @substitute!(%)( \delbar{\del{X_{\mu}}} -> \pdel{\delbar{X_{\mu}}} ):
▷ @pintegrate!(%){ \pdelbar }:
▷ @pintegrate!(%){ \pdel }:
▷ @rename!(%){"\pdelbar"}{"\delbar"}:
▷ @rename!(%){"\pdel"}{"\del"};
▷ @prodrule!(%);
▷ @distribute!(%);
▷ @unwrap!(%);
▷ @sort_product!(%);
```

Notice how, after the partial integration, we renamed the partially integrated derivatives back to normal ones (and again apply Leibnitz' rule). If we now collect terms,

```
▷ @collect_terms!(%);
```

we indeed find that the total susy variation vanishes.

2.5 Tutorial 4: Super-Maxwell

The following example illustrates the use of a somewhat more complicated set of object properties. A TEXmacs version of this problem can be found in the distribution tarball in the file `texmacs/showcase3.tm`. We start with the super-Maxwell action, given by

$$S = \int \mathrm{d}^4x \Big[-\frac{1}{4}(F_{ab})^2 - \frac{1}{2}\bar{\lambda}\gamma^a\partial_a\lambda \Big] , \tag{2.4}$$

It is supposed to be invariant under the transformations

$$\delta A_a = \bar{\epsilon}\gamma_a\lambda , \quad \delta\lambda = -\frac{1}{2}\gamma^{ab}\epsilon\, F_{ab} . \tag{2.5}$$

The object properties for this problem are

```
▷ { a,b,c,d,e }::Indices(vector).
▷ \bar{#}::DiracBar.
▷ { \partial{#}, \ppartial{#} }::PartialDerivative.
▷ { A_{a}, f_{a b} }::Depends(\partial, \ppartial).
▷ { \epsilon, \gamma_{#} }::Depends(\bar).
▷ \lambda::Depends(\bar, \partial).
```

```
▷ { \lambda, \gamma_{#} }::NonCommuting.
▷ { \lambda, \epsilon }::Spinor(dimension=4, type=Majorana).
▷ { \epsilon, \lambda }::SortOrder.
▷ { \epsilon, \lambda }::AntiCommuting.
▷ \lambda::SelfAntiCommuting.
▷ \gamma_{#}::GammaMatrix(metric=\delta).
▷ \delta{#}::Accent.
▷ f_{a b}::AntiSymmetric.
▷ \delta_{a b}::KroneckerDelta.
```

Note the use of two types of properties: those which apply to a single object, like `Depends`, and those which are associated to a list of objects, like `AntiCommuting`. Clearly $\partial_a\lambda$ and $\bar{\epsilon}$ are anti-commuting too, but the program figures this out automatically from the fact that `\partial` has an associated `PartialDerivative` property associated to it.[3]

The actual calculation is an almost direct transcription of the formulas above. First we define the supersymmetry transformation rules,

```
▷ susy:= { \delta{A_{a}} = \bar{\epsilon} \gamma_{a} \lambda,
           \delta{\lambda} = -(1/2) \gamma_{a b} \epsilon f_{a b} };
```

The action is also written just as it is typed in TEX,

```
▷ S:= -(1/4) f_{a b} f_{a b}
                - (1/2) \bar{\lambda} \gamma_{a} \partial_{a}{\lambda};
```

Showing invariance starts by applying a variational derivative,

```
▷ @vary(%)( f_{a b} ->   \partial_{a}{\delta{A_{b}}}
                       - \partial_{b}{\delta{A_{a}}},
            \lambda -> \delta{\lambda} );

▷ @distribute!(%):
▷ @substitute!(%)( @(susy) ): @prodrule!(%): @distribute!(%): @unwrap!(%);
```

After these steps, the result is (shown exactly as it appears in the TEXmacs [**?**] frontend)

$$S = \bar{\epsilon}\gamma_a\partial_b\lambda\, f_{ab} + \frac{1}{4}\overline{\gamma_{cb}\epsilon}\gamma_a\partial_a\lambda\, f_{cb} + \frac{1}{4}\bar{\lambda}\gamma_a\gamma_{cd}\epsilon\partial_a f_{cb}\,. \tag{2.6}$$

Since the program knows about the properties of gamma matrices it can rewrite the Dirac bar, and then we do one further partial integration,

```
▷ @rewrite_diracbar!(%);
▷ @substitute!(%)( \partial_{c}{f_{a b}} -> \ppartial_{c}{f_{a b}} ):
▷ @pintegrate!(%){\ppartial}:
▷ @rename!(%){"\ppartial"}{"\partial"}:
▷ @prodrule!(%): @unwrap!(%);
```

[3] This is similar to Macsyma's types and features: the property which is attached to a symbol is like a 'type', while all properties which the symbol inherits from child nodes are like 'features'.

What remains is the gamma matrix algebra, sorting of spinors (which employs inheritance of the `Spinor` and `AntiCommuting` properties as already alluded to earlier) and a final canonicalisation of the index contractions,

```
▷ @join!(%){expand}: @distribute!(%): @eliminate_kr!(%): @sort_product!(%);
▷ @substitute!(%)( \partial_{a}{\bar{\lambda}}
                                    -> \bar{\partial_{a}{\lambda}} );
▷ @spinorsort!(%):
▷ @rename_dummies!(%): @canonicalise!(%): @collect_terms!(%);
```

The result is a Bianchi identity on the field strength, and thus invariance of the action.

Graphical user interface

3.1 General information

The graphical user interface is called `xcadabra`.

Context-sensitive help is available: if you put your cursor on a command or property name and press F1 or the help button, you will get a help screen (which is essentially the corresponding section in this manual).

3.2 Cell types

There are *two* input cell types and *two* output cell types. Normal input cells contain cadabra input. TEX input cells contain comments which you can add to the notebook but which will not be sent to cadabra. Output cells can be either verbatim output, such as status messages, or TEX output of expressions. Output cells are associated to input cells, so if you delete an input cell all the associated output cells will be removed too.

In the current version there is no visual separation between cells. When you start a new blank notebook, your cursor is in the first input cell. After entering an expression there and pressing `shift-enter`, this input cell will be evaluated and all output will be placed below it in the first output cell. In addition, a new input cell will be created below the output cell.

The "edit" menu can be used to add input cells or delete them, and also shows keyboard shortcuts.

3.3 Editing, loading, saving, printing, exporting

The file format used by the front-end is a LATEX compatible file. If necessary, it can be edited by hand using a text editor.

As the file format which the front-end uses to save notebooks is a LATEX compatible file, it is simple to print a notebook: simply save it and run it through LATEX as usual. You will need two special-purpose packages, `breqn` (which is separately packaged and also available from the AMS) and `tableaux` (which is included with the cadabra distribution).

Notebooks can be exported to a normal cadabra text input file. This results in a file which contains only the input cells of the notebook. Such a file can be fed into the command line version of cadabra by using

```
▷ cadabra --input [filename]
```

The default extension for cadabra notebooks is `.cnb`. For cadabra input files (for the command line version of cadabra) it is `.cdb`.

3.4 Evaluating cells

In order to evaluate cells, press `shift-enter` in an input cell. This will send the input to the kernel and display any results or informal messages below the input cell.

3.5 Algorithm and property auto-completion

The GUI has shell-like automatic completion of algorithm and property names. If you type only part of an algorithm or property, and subsequently press the `TAB` key, the name will be completed to in maximally unambiguous way. For instance, by typing `::Com` and hitting the `TAB` key, the input will be completed to `::Commuting`. By adding `AsP` and hitting TAB again, this gets completed to `::CommutingAsProduct`.

3.6 Cutting and pasting

You can select output cells and paste them back into input cells or into some other application. The program offers two formats for pasting, a LaTeX one which is useful to paste into a paper, and a cadabra one which represents the way in which the expression can be pasted back into the program.

In most cases, the format will be selected correctly automatically. Under Emacs, a middle-mouse paste will paste the LaTeX version. You can paste the internal cadabra format by adding the following to your `.emacs` file:

```
(defun pastecdb ()
   (interactive)
   (insert (x-get-selection-internal 'PRIMARY 'cadabra)))

(global-set-key (kbd "C-x p") 'pastecdb)
```

This will make the combination "Ctrl-x p" insert the current selection in cadabra internal format.

For more advanced users: the list of formats can be inspected using

```
▷ (x-get-selection-internal 'PRIMARY 'TARGETS)
  [TIMESTAMP TARGETS MULTIPLE cadabra UTF8_STRING TEXT]
```

If e.g. an output cell with the content `1/2 A_{m}^{n}` is selected, the two different paste formats can be obtained by evaluating the following two LISP expressions in a scratch buffer,

```
▷ (x-get-selection-internal 'PRIMARY 'cadabra)
  #("1/2 A_{m}^{n};" 0 14 (foreign-selection STRING))
```

```
▷ (x-get-selection-internal 'PRIMARY 'TEXT)
  #("1 := \frac{1}{2}\, A_{m}\,^{n};" 0 31 (foreign-selection COMPOUND_TEXT))
```

Note that the TEXT format contains markup such as spacing commands which make it more useful for papers. This is the default format when the middle mouse button is pressed.

Reference guide

4.1 Basics about the input format

The input format of cadabra is closely related to the notation used by TEX to denote tensorial expressions. That is, one can use not only bracketed notation to denote child objects, like in

```
object[child,child]
```

but also the usual sub- and superscript notation like

```
object^{child child}_{child}
```

One can use backslashes in the names of objects as well, just as in TEX. All of the symbols that one enters this way are considered "passive", that is, they will go into the expression tree just like one has entered them.

"Active nodes", on the other hand, are symbols pre-fixed with a "@" symbol. These are nodes which lead to an algorithm being applied to their children; they will thus not end up in the actual tree stored in memory, but instead get replaced with the output that their algorithm produces (if any). Finally, every non-empty expression that one enters will labelled by a number. Additionally, one can give an expression a label by entering it as "`label-text:= expression;`". One can use either the expression number or the label to refer to an expression.

Names of objects can contain any printable character, except for brackets, sub- and super-script symbols and backslash characters (the latter can only occur at the beginning of a name). The sole exception to this rule is the names of active nodes: these names *can* contain underscores to separate words. The program can also deal with "accents" added to symbol names, like

```
\hat{A}
\bar{B}
\prime{Q}
```

The precise way in which this works is explained in section **??**.

Input lines always have to be terminated with either a ";", a ":" or a ".". The first two of these delimiting symbols act in the same way as in Maple: the second form suppresses the output of the entered expression (see section **??** for additional information on how to redirect output to files). The last delimiter has the effect of evaluating the resulting expression, and immediately discarding it, also disabling printing. Long expressions can, because of these delimiters, be spread over many subsequent input lines. Any line starting with a "#" sign is considered to be a comment (even when it appears within a multi-line expression). Comments are always ignored completely (they do not end up in the expression tree).

All internal storage is in prefix notation, but a converter is applied to the input which transforms normal infix notation for products and sums to prefix notation.

4.2 Active nodes or "commands"

Active nodes are nodes which lead to immediate evaluation after the input has been entered; these could thus be called "commands". Commands always start with a "@" symbol, to distinguish them from normal, unevaluated input. Used in the standard "notebook" way, active nodes act on their first argument and get replaced with the result that they compute. This form uses square brackets to denote the argument on which to act:

```
@command[expression]{arg1}{arg2}...{argn}
```

If you want to re-use an existing expression as the argument, this can be cloned by using the "@" operator itself:

```
@[equation number or label]
```

or (for convenience, as a single exception to the square bracket rule)

```
@(equation number or label)
```

both evaluate to a copy of the expression to which they refer (one can use the "%" character to denote the number of the last-used expression). Note that all of these forms create a *new* expression. One is advised *not* to refer to equation numbers using this command, as this makes expressions tied together in a rather fragile way.

Algorithms can also be made to act on existing expressions such as to modify the expression history (i.e. in the language of the previous section, to stay within a "mini-notebook"). This is done by using round brackets instead of square ones:

```
@command(expression number or label){arg1}{arg2}...{argn}
```

Here it is no longer problematic to refer to equation numbers, as the result of this command will appear in the history list of the indicated expression (and is therefore not tied to the particular number of the expression).

All of these commands act on the top of the argument subtree. You can make them act subsequently on all nodes to which they apply by postfixing the name with an exclamation mark, as in

```
@command![expression]{arg1}{arg2}...{argn}
```

This will search the tree in pre-order style, applying the algorithm on every node to which the algorithm applies.

If you want an algorithm to act until the expression no longer changes, use a double exclamation mark. Compare the single-exclamation mark version,

```
▷ A_{m} B_{m} A_{n} B_{n};
▷ @substitute!(%)( A_{m} B_{n} -> Q);
  Q A_{n} B_{n};
```

with the double-exclamation mark one,

```
▷ A_{m} B_{m} A_{n} B_{n};
▷ @substitute!!(%)( A_{m} B_{n} -> Q );
  Q Q;
```

Be careful with this functionality, as there is currently no safeguard in case the expression keeps changing forever.

Instead of acting only at the top of the argument subtree, or acting on all nodes individually, it is also possible to act only a specific level of the expression. The notation for this is an exclamation mark with an extra number attached to it, which indicates the level (starting at 1 for the highest level),

```
@command!level(expression number of label){arg1}{arg2}...{argn}
```

Note that `\sum` and `\prod` nodes in the expression tree also count as a level.

4.3 Object properties and declaration

Symbols in cadabra have no a-priori "meaning". If you write `\Gamma`, the program will not know that it is supposed to be, for instance, a Clifford algebra generator. You will have to declare the properties of symbols, i.e. you have to tell cadabra explicitly that if you write `\Gamma`, you actually mean a Clifford algebra generator. This indirect way of attaching a meaning to a symbol has the advantage that you can use whatever notation you like; if you prefer to write `\gamma`, or perhaps even `\rho` if your paper uses that, then this is perfectly possible (object properties are a bit like "attributes" in `Mathematica` or "domains" in `Axiom` and `MuPAD`).

Properties are all written using capitals to separate words, as in `AntiSymmetric`. This makes it easier to distinguish them from commands. Properties of objects are declared by using the "::" characters. This can be done "at first use", i.e. by just adding the property to the object when it first appears in the input. As an example, one can write

```
▷ F_{m n p}::AntiSymmetric;
```

This declares the object to be anti-symmetric in its indices and at the same time creates a new expression with this object. The property information is stored separately, so that further appearances of the "`F_{m n p}`" object will automatically share this property. It is of course also possible to keep object property declarations separated (for instance at the top of an input file). This is done as follows:

```
▷ F_{m n p}::AntiSymmetric.
▷ F_{m n p};
```

A list of all properties is available in the graphical interface through the Help menu, and can also be obtained using the `@properties` command.

Note that properties are attached to patterns. Therefore, you can have

```
▷ R::RicciScalar.
▷ R_{m n p q}::RiemannTensor.
```

at the same time. The program will not warn you if you use incompatible properties, so if you make a declaration like above and then later on do

```
▷ R::Coordinate.
```

this may lead to weird results.

The fact that objects are attached to patterns also means that you can use something like wildcards. In the following declaration,

```
▷ { m#, n# }::Indices(vector).
```

the entire infinite set of objects $m1, m2, m3, \ldots$ and $n1, n2, n3, \ldots$ are declared to be in the dummy index set "vector".[4] Range wildcards can also be used to match one or more objects of a specific type. In this case, they always start with a "#" sign, followed by optional additional information to specify the type of the object and the number of them. The declaration

```
{m,n,p,q,r,s}::Indices(vector).
A_{ #{m, 1..3} }::AntiSymmetric.
B_{ #{m} }::Symmetric.
```

indicates that whenever the object `A` appears with one to three indices of the vector type, it is antisymmetric. If the index type does not match, or if there are zero or more than three indices, the property does not apply. Similarly, `B` is always symmetric when all of its indices are of the vector type.

Properties can be assigned to an entire list of symbols with one command, namely by attaching the property to the list. For example,

```
▷ {n, m, p, q}::Integer(1..d).
```

This associates the property "Integer" to each and every symbol in the list. However, there is also a concept of "list properties", which are properties which are associated to the list as a whole. Examples of list properties are "AntiCommuting" or "Indices". See for a discussion of list properties section **??**.

Objects can have more than one property attached to them, and one should therefore not confuse properties with the "type" of the object. Consider for instance

```
▷ x::Coordinate.
▷ W_{m n p q}::WeylTensor.
▷ W_{m n p q}::Depends(x).
```

[4] This way of declaring ranges of objects is similar to the "autodeclare" declaration method of FORM [**?**].

This attaches two completely independent properties to the pattern W_{mnpq}.

In the examples above, several properties had arguments (e.g. "`vector`" or "`1..d`"). The general form of these arguments is a set of key-value pairs, as in

```
▷ T_{m n p q}::TableauSymmetry(shape={2,1}, indices={0,2,1}).
```

In the simple cases discussed so far, the key and the equal sign was suppressed. This is allowed because one of the keys plays the role of the default key. Therefore, the following two are equivalent,

```
▷ { m, n }::Integer(range=0..d).
▷ { m, n }::Integer(0..d).
```

See the detailed documentation of the individual properties for allowed keys and the one which is taken as the default.

Finally, there is a concept of "inherited properties". Consider e.g. a sum of spinors, declared as

```
▷ {\psi1, \psi2, \psi3}::Spinor.
▷ \psi1 + \psi2 + \psi3;
```

Here the sum has inherited the property "Spinor", even though it does not have normal or intrinsic property of this type. Properties can also inherit from each other, e.g.

```
▷ \Gamma_{#}::GammaMatrix.
▷ \Gamma_{p o i u y};
▷ @indexsort!(%);
```

The `GammaMatrix` property inherits from `AntiSymmetric` property, and therefore the `\Gamma` object is automatically anti-symmetric in its indices.

A list of all properties known to cadabra can be obtained by using the `@proplist` command.

4.4 List properties and symbol groups

Some properties are not naturally associated to a single symbol or object, but have to do with collections of them. A simple example of such a property is `AntiCommuting`. Although it sometimes makes sense to say that "ψ_m is anticommuting" (meaning that $\psi_m\psi_n = -\psi_n\psi_m$), it happens just as often that you want to say "ψ and χ anticommute" (meaning that $\psi\chi = -\chi\psi$). The latter property is clearly relating two different objects.

Another example is dummy indices. While it may make sense to say that "m is a dummy index", this does not allow the program to substitute m with another index when a clash of dummy index names occurs (e.g. upon substitution of one expression into another). More useful is to say that "m, n, and p are dummy indices of the same type", so that the program can relabel a pair of m's into a pair of p's when necessary.

In cadabra such properties are called "list properties". You can associate a list property to a list of symbols by simply writing, e.g. for the first example above,

```
▷ { \psi, \chi }::AntiCommuting.
```

Note that, as described in section **??**, you can also attach normal properties to multiple symbols in one go using this notation. The program will figure out automatically whether you want to associate a normal property or a list property to the symbols in the list.

Lists are ordered, although the ordering does not necessarily mean anything for all list properties (it is relevant for e.g. `SortOrder` but irrelevant for e.g. `AntiCommuting`).

4.5 Indices, dummy indices and automatic index renaming

In cadabra, all objects which occur as subscripts or superscripts are considered to be "indices". The names of indices are understood to be irrelevant when they occur in a pair, and automatic relabelling will take place whenever necessary in order to avoid index clashes.

Cadabra knows about the differences between free and dummy indices. It checks the input for consistency and displays a warning when the index structure does not make sense. Thus, the input

```
▷ A_{m n} + B_{m} = 0;
```

results in an error message.

The location of indices is, by default, not considered to be relevant. That is, you can write

```
▷ A_{m} + A^{m};
▷ A_{m} B_{m};
```

as input and these are considered to be consistent expressions. If, however, the position of an index means something (like in general relativity, where index lowering and raising implies contraction with a metric), then you can declare index positions to be "fixed". This is done using

```
▷ {m, n, p}::Indices(position=fixed).
```

When substituting an expression into another one, dummy indices will automatically be relabelled when necessary. To see this in action, consider the following example:

```
▷ {p,q}::Indices(vector).
▷ F_{m n} F^{m n};
  F_{m n} F^{m n}

▷ G_{m n} @(1);
  G_{m n} F_{p q} F^{p q}
```

The m and n indices have automatically been converted to p and q in order to avoid a conflict with the free indices on the G_{mn} object.

Refer to section **??** for commands that deal with dummy indices.

argument	whitespace	dummy	example
{}	product		`\frac{a b}{c} = \frac{a * b}{c}`
()	product		`sin(a b) = sin(a * b)`
_{}	separator	yes	`M_{a b} = M_{a}_{b}`
^{}	separator	yes	`M^{a b} = M^{a}^{b}`
_()	separator		`M_(a b) = M_(a)_(b)`
^()	separator		`M^(a b) = M^(a)^(b)`
[]	product		`[a b, c] = [a * b, c]`

Table 1: The implicit meaning of objects and whitespace inside the various types of brackets.

4.6 Exponents, indices and labels

Exponents, indices and labels are traditionally all placed using super- or subscript notation, making these slightly ambiguous for a computer algebra system. Since this situation can become quite complex (how do you represent the third power of the first element of a vector?) cadabra requires all powers to be entered with a double-star "$**$" notation:[5]

```
▷ a**2;
▷ a**(2 + 3b);
```

In addition, all sub- or superscripts with *curly* braces indicate indices, to which the summation convention applies. In particular, there are not allowed to be more than two identical indices in a single product. The summation convention does not apply to arguments with any other bracket types. In particular, sub- or superscripts with *square* or *pointy* brackets have (as of yet) no fixed meaning.

4.7 Spacing and brackets

Cadabra is reasonably flexible as far as spacing and brackets are concerned, but the fact that objects do not have to be declared before they can be used means that spaces have to be introduced in some cases to avoid ambiguity. As a general rule, all terms in a product have to be separated by at least one whitespace character. Thus,

`A(B+C)` incorrect, (interpreted as `A` with argument `B+C`),

`AB` incorrect, (interpreted as one object, not two)

`A (B+C)` correct,

`A*(B+C)` correct.

If a whitespace character is absent, all brackets are interpreted as enclosing *argument* groups. Products of variables (e.g. AB) have to be separated by a space, otherwise the input will be read as the name of a single variable. However, spaces will automatically be

[5] Solutions involving the declaration of which indices are exponents and which are indices have been considered, but rejected in favour of the explicit "$**$" notation. Cases like "the square of the second component of a vector" quickly become ambiguous even with rather complicated property declarations, while the "$**$" notation remains transparent.

inserted after numbers, between a closing bracket and a name or number, and between two names if the second name starts with a backslash. The following expressions are therefore interpreted as one would expect:

`3A`	interpreted as `3*A`
`(A+B)C`	interpreted as `(A+B)*C`
`(A+B)3`	interpreted as `(A+B)*3`
`A\Gamma`	interpreted as `A*`Γ

This conversion is done by the preprocessor. Finally, note that brackets in the input *must* be balanced (a decision made to simplify the parser; it means that cadabra uses a different way to indicate groups of symmetric or anti-symmetric indices than one often encounters in the literature; see section **??**).

4.8 Implicit versus explicit indices

When writing expressions which involves vectors, spinors and matrices, one often employs an implicit notation in which some or all of the indices are suppressed. Examples are

$$a = Mb\,, \qquad \bar{\psi}\gamma^m\chi\,, \tag{4.1}$$

where a and b are vectors, ψ and χ are spinors and M and γ^m are matrices. Clearly, the computer cannot know this without some further information. In cadabra objects can carry implicit indices, through the `ImplicitIndex` property. There are derived forms of this, e.g. `Matrix` and `Spinor`,

```
▷ {a,b}::ImplicitIndex;
▷ M::Matrix.
▷ a = M b;
▷ @sort_product!(%);
  @sort_product: not applicable.
```

If you had not made the property assignment in the first two lines, the `@sort_product` would have incorrectly swapped the matrix and vector, leading to a meaningless expression.

If you have more than one set of implicit indices, you can label them just like you can label explicit indices,

```
▷ {a,b}::ImplicitIndex(type1);
▷ {c,d}::ImplicitIndex(type2);
▷ M::Matrix(type1).
▷ a = M d c b;
▷ @sort_product!(%);
  a = M b d c;
```

4.9 Index brackets

Indices can be associated to tensors, as in $T_{\mu\nu}$, but it often also happens that we want to associate indices to a sum or product of tensors, without writing all indices out explicitly. Examples are

$$(A + B + C)_{\alpha\beta}\,, \quad \text{or} \quad (\psi\Gamma_{mn}\Gamma_p)_\beta\,. \tag{4.2}$$

Here the objects A, B, C and Γ are matrices, while ψ is a vector. Their explicit components are labelled with α and β indices, but the notation above keeps most of these vector indices implicit.

Cadabra can deal with such expressions through a construction which is called the "indexbracket". It is possible to convert from one form to the other by using the `@combine` and `@expand` algorithms. Combining terms goes like this,

```
▷ (\Gamma_r)_{\alpha\beta} (\Gamma_{s t u})_{\beta\gamma}:
▷ @combine!(%);
  (\Gamma_r \Gamma_{s t u})_{\alpha\gamma};
```

or as in

```
▷ (\Gamma_r)_{\alpha\beta} Q_\beta:
▷ @combine!(%);
  (\Gamma_r Q)_{\alpha};
```

If the index bracket has only one index, either the first or the last argument should be a matrix, but not both:

```
▷ A::Matrix.
▷ {m,n,p}::Indices(vector).
  (A B)_{m};
▷ @expand(%);
  A_{m n} B_{n};
```

If the index bracket has two indices, all arguments should be matrices,

```
▷ {A,B}::Matrix.
▷ {m,n,p}::Indices(vector).
▷ (A B)_{m n};
  @expand(%);
  A_{m p} B_{p n};
```

If there are more arguments inside the bracket, these of course all need to be matrices (and are assumed to be so by default).

4.10 Derivatives and implicit dependence on coordinates

There is no fixed notation for derivatives; as with all other objects you have to declare derivatives by associating a property to them, in this case the `Derivative` property.

```
▷ \nabla{#}::Derivative.
```

Derivative objects can be used in various ways. You can just write the derivative symbol, as in

```
▷ \nabla{ A_{\mu} };
```

(a notation used e.g. in the tutorial example in section **??**). Or you can write the coordinate with respect to which the derivative is taken,

```
▷ s::Coordinate.
▷ A_{\mu}::Depends(s).
▷ \nabla_{s}{ A_{\mu} };
```

Finally, you can use an index as the subscript argument, as in

```
▷ { \mu, \nu }::Indices(vector).
▷ \nabla_{\nu}{ A_{\mu} };
```

(in which case the first line is, for the purpose of using the derivative operator, actually unnecessary).

The main point of associating the `Derivative` property to an object is to make the object obey the Leibnitz or product rule, as illustrated by the following example,

```
▷ \nabla{#}::Derivative.
▷ \nabla{ A_{\mu} * B_{\nu} };
▷ @prodrule!(%);
  \nabla{A_{\mu}} B_{\nu} + A_{\mu} \nabla{B_{\nu}};
```

This behaviour is a consequence of the fact that `Derivative` derives from `Distributable`. Note that the `Derivative` property does not automatically give you commuting derivatives, so that you can e.g. use it to write covariant derivatives.

More specific derivative types exist too. An example are partial derivatives, declared using the `PartialDerivative` property. Partial derivatives are commuting and therefore automatically symmetric in their indices,

```
▷ \partial{#}::PartialDerivative.
▷ {a,b,m,n}::Indices(vector).
▷ C_{m n}::Symmetric.

▷ T^{b a} \partial_{a b}( C_{m n} D_{n m} );
▷ @canonicalise!(%);
  T^{a b} \partial_{a b}( C_{m n} D_{m n} );
```

4.11 Accents

It often occurs that you want to put a hat or tilde or some other accent on top of a symbol, as a means to indicate a slightly different object. In most of these cases, the properties of the normal symbol and the accented symbol are identical. Such accents are declared using the `Accent` property, as in

```
\hat{#}::Accent.
```

This automatically makes all symbols with hats inherit the properties of the unhatted symbols,

```
▷ \hat{#}::Accent.
▷ {\psi, \chi}::AntiCommuting.
▷ {\psi, \chi}::SortOrder.
▷ \hat{\chi} \psi:
▷ @sort_product!(%);
  (-1) \psi \hat{\chi};
```

If you want to put an accent on an object with indices, wrap the accent around the entire object, do not leave the indices outside.

Note that it is also possible to mark objects by attaching sub- or superscripted symbols to them, as in e.g. $A^\dagger$. This can be done by declaring these symbols explicitly using the `Symbol` property,

```
▷ \dagger::Symbol.
```

If you do not do this, `\dagger` will be seen as an index and an error will be reported if it appears more than twice.

4.12 Symbol ordering and commutation properties

A conventional way to sort factors in a product is to use lexographical ordering. However, this is almost never what one wants when transcribing a physics problem to the computer. Therefore, cadabra allows you to specify the sort order of symbols yourself. This is done by associating the `SortOrder` list property to a list of symbols, as in

```
▷ {X,G,Y,A,B}::SortOrder.
▷ A*B*G*X*A*X:
▷ @sort_product(%);
  X*X*G*A*A*B;
```

More complicated objects with indices are of course also allowed, such as in

```
▷ { W_{m n}, W_{m} }::SortOrder.
▷ W_{m n} W_{p} W_{q r} W_{s} W_{t}:
▷ @sort_product(%);
  W_{m n} * W_{q r} * W_{p} * W_{s} * W_{t};
```

For the time being, it is not allowed to have more than one such set contain the same symbol. Thus,

```
▷ {X,G}::SortOrder.
▷ {X,A,B}::SortOrder.
```

is not allowed (and will, in fact, take X out of the first list).

Apart from the preferred sort order, there are more properties which influence the way in which products can be sorted. In particular, sort order is influenced by whether symbols commute or anti-commute with each other. Physicists in general use a very implicit notation as far as commutativity of objects in a product is concerned. Consider for instance a situation in which we deal with two operators $\hat{M}$ and $\hat{N}$, as well as some constants q and p. These two expressions are equivalent:

$$2q\,\hat{M}p\hat{N} \quad \text{and} \quad 2pq\,\hat{M}\hat{N}\,. \tag{4.3}$$

But this is not obvious from the notation that has been used to indicate the product. In fact, the product symbol is usually left out completely.

In many other computer algebra systems, you have to introduce special types of "non-commuting" products (e.g. the `&*` operator in Maple or the `**` operator in Mathematica). This can be rather cumbersome, for a variety of reasons. The main reason, however, is that it does not match with what you do on paper. On paper, you never write special product symbols for objects which do not commute. You just string them together, and know from the properties of the symbols whether objects can be moved through each other or not.

In order to make these sort of things possible in cadabra, it is necessary to declare "sets" of objects which mutually do not commute (i.e. for which the order inside a product cannot be changed without consequences) but which commute with objects of other sets. Many computer algebra systems only use one such set: objects are either "commuting" or "non-commuting". This is often too restrictive. For instance, when Ψ is a fermion and Γ denotes a Clifford algebra element, a system with only one set of non-commuting objects is unable to see that

$$\bar{\Psi}_a(\Gamma_n\Gamma_m)_{ab}\Psi_b \quad \text{and} \quad \bar{\Psi}_a\Psi_b(\Gamma_n\Gamma_m)_{ab} \tag{4.4}$$

are equivalent. In cadabra, one would simply put Ψ_a and Γ_m in two different sets, mutually commuting, but non-commuting among themselves. To be precise, the example above is reproduced by

```
▷ \bar{#}::Accent.
▷ \Psi_{a}::SelfNonCommuting.
▷ \Gamma_{#}::GammaMatrix.
▷ \bar{\Psi_a} (\Gamma_n \Gamma_m)_{ab} \Psi_b:
▷ @sort_product(%);
  \bar{\Psi_a} \Psi_b (\Gamma_n \Gamma_m)_{ab};
```

(see section **??** for an explanation of the `Accent` property).

Commutation properties always refer to components. If you associate an `ImplicitIndex` property to an object, then it will never commute with itself or with any other such object. You can see this in the example above, as the `@sort_product` command kept the order of the two gamma matrices unchanged.

4.13 Anti-commuting objects and derivatives

We have discussed anti-commuting objects to some extend in the previous section, but things become somewhat more hairy in combination with derivatives. Hence the current section, in which we will look in more detail at how to write properties which take care of anti-commuting derivatives (as they occur in e.g. superspace field theories).

We will use the `@prodrule` command to show how the various property assignments work.

There are various ways in which you may want to write derivatives which are anti-commuting. One way is to use explicit indices, and declare them to be anti-commuting, as in

```
▷ {\alpha,\beta,\gamma}::Indices(spinor).
▷ {\alpha,\beta,\gamma}::AntiCommuting.
▷ D{#}::Derivative.
▷ D_{\alpha}{\theta^\beta \theta^\gamma};
  D_{\alpha}{\theta^{\beta}} \theta^{\gamma}
             - \theta^{\beta} D_{\alpha}{\theta^{\gamma}};
```

Note the appearance of the minus sign for the second term. Writing things in this way is the most straightforward way to handle anti-commuting derivatives, and only requires the assignment of the `AntiCommuting` property to the indices (second line above).

However, one often deals with implicit indices when tackling problems involving anti-commuting variables. Here is an example to illustrate what is meant by this:

```
▷ test 19d does not do the right thing: it does not pick up a minus sign
▷ at the first step.
```

4.14 Input and output redirection

By default, all generated output is displayed on the screen. As mentioned in section **??**, the delimiter of a line determines whether output is suppressed or not. It is, however, also possible to write output to a file, by appending a file name to the line delimiter, as in

```
▷ a^2+b^2 = c^2; "output_file"
```

(note that there is no semi-colon following the file name). This shows the filename "output_file" on the display, and writes to this file the expression

```
1:=  a^{2}+b^{2}=c^{2}
```

This is exactly as it would have appeared on the display. If you need a different kind of output, use the algorithms provided in the `output` module (see section **??**).

It is also possible to read input from files instead of from the terminal:

```
▷ < "input_file"
```

reads from the file "input_file" until the end or until a line `@end;` is found, whichever comes first. Input redirect can be nested. In case an error occurs, reading from the file is also aborted and input is taken from the terminal again.

4.15 Default rules

By default, cadabra does very few things "by itself" with your expressions. This goes as far as not collecting equal terms, so that you can write

```
▷ A + A;
```

and have this stored as a sum of two terms. However, you can add an arbitrary number of default commands, which are run just after you have entered an expression on the command line (`PreDefaultRules`) or just before the output is returned to you (`PostDefaultRules`). So if you want terms to be collected automatically, just enter

```
▷ ::PostDefaultRules( @@collect_terms!(%) ).
```

Note the double "@@" symbol, which prevents immediate execution of the command. This post default rule will collect equal terms before they ever make it to internal storage.

Many more complicated things can be achieved with this feature. You can, for instance, also use complicated substitution commands as arguments to `PreDefaultRules` or `PostDefaultRules`. An example is

```
▷ {m,n,p,q}::Indices(vector).
▷ ::PreDefaultRules( @@substitute!(%)( A_{m n} -> B_{m q} B_{q n} ).

▷ A_{m n} A_{n m};
```

which immediately returns

```
B_{m q} B_{q n} B_{n p} B_{p m};
```

As usual dummy indices have been relabelled appropriately.

4.16 Patterns, conditionals and regular expressions

Patterns in cadabra are quite a bit different from those in other computer algebra systems, because they are more tuned towards the pattern matching of objects common in tensorial expressions, rather than generic tree structures.

Name patterns are constructed by writing a single question mark behind the name, as in

```
▷ @substitute!(%)( A? + B? -> 0 );
```

which matches all sums with two terms, each of which is a single symbol without indices or arguments. If you want to match instead any object, with or without indices or arguments, use the double question mark instead. To see the difference more explicitly, compare the two substitute commands in the following example:

```
▷ A_{m n} + B_{m n}:
▷ @substitute!(%)( A? + B? -> 0 );
  A_{m n} + B_{m n};
▷ @substitute!(%)( A?? + B?? -> 0 );
  0;
```

Note that it does not make sense to add arguments or indices to object patterns; a construction of the type "`A??_{\mu}(x)`" is meaningless and will be flagged as an error.

There is a special handling of objects which are dummy objects in the classification of section **??** (see table **??**). Objects of this type do not need the question mark, as their explicit name is never relevant. You can therefore write

```
▷ @substitute!(%)( A_{m n} -> 0 );
```

to set all occurrances of the tensor A with two subscript indices to zero, regardless of the names of the indices (i.e. this command sets A_{pq} to zero).

When replacing object wildcards with something else that involves these objects, use the question mark notation also on the right-hand side of the rule. For instance,

```
▷ @substitute!(%)( A? + B? -> A? A? );
```

replaces all sums with two elements by the square of the first element. The following example shows the difference between the presence or absence of question marks on the right-hand side:

```
▷ C + D:
▷ @substitute!(%)( A? + B? -> A? A? );
  C * C;
▷ @pop(%);
▷ @substitute!(%)( A? + B? -> A A );
  A * A;
```

Note that you can also use patterns to remove or add indices or arguments, as in

```
▷ {\mu,\nu,\rho}::Indices(vector).
▷ A_{\mu} B_{\nu} C_{\nu} D_{\mu};
▷ @substitute!!(%)( A?_{\rho} B?_{\rho} -> \dot(A?)(B?) );
  A.D B.C;
▷ @substitute!!(%)( \dot(A?)(B?) -> A?_{\mu} B?_{\mu} );
```

In many algorithms, patterns can be supplemented by so-called conditionals. These are constraints on the objects that appear in the pattern. For instance, the substitution command

```
▷ @substitute!(%)( A_{m n} B_{p q} | _{n}!=_{p} -> 0 );
```

applies the substitution rule only if the second index of A and the first index of B do not match. Note that the conditional follows directly after the pattern, not after the full substitution rule. A way to think about this is that the conditional is part of the pattern, not of the rule. The reason why the conditional follows the full pattern, and not directly the symbol to which it relates, is clear from the example above: the conditional is a "global" constraint on the pattern, not a local one on a single index.

These conditions can be used to match names of objects using regular expressions. Consider the following example:

```
▷ A + B3 + C7:
▷ @substitute!(%)( A + M? + N?  | { \regex{M?}{"B[0-9]*"},
                                  \regex{N?}{"[A-Z]7"} } -> sin(M? N?)/N? );
  sin(B3 C7)/C7;
```

It is also possible to select objects based on whether or not they are associated with a particular property, e.g.[6]

```
▷ A::SelfAntiCommuting.
▷ A A + B B;
▷ @substitute!(%)( Q?? Q?? | \hasprop{Q??}{SelfAntiCommuting} -> 0 );
  B B;
```

Combinations of different types of conditionals are of course also possible. This type of constraint can be used to construct replacement rules which act on objects of a certain type without knowing what their name is. The following example illustrates this.

```
▷ \sigma_{a?}::SigmaMatrix.
▷ \sigma_{1}\sigma_{2} + \rho_{1} \rho_{2};
▷ @substitute!(%)( S?_{1} S?_{2} | \hasprop{S?_{1}}{SigmaMatrix} -> I*S?_{3}  );
  I \sigma_{3} + \rho_{1} \rho_{2};
```

This could also have been written without specifying the numerical values of the indices explicitly,

```
▷ \sigma_{a?}::SigmaMatrix.
▷ \sigma_{1}\sigma_{2} + \rho_{1} \rho_{2};
▷ @substitute!(%)( S?_{a?} S?_{b?} | \hasprop{S?_{a?}}{SigmaMatrix} -> I*\epsilon_{a? b? c} \sig
  I \epsilon_{1 2 c} \sigma_{c} + \rho_{1} \rho_{2};
```

[6] More compact notations may look tempting, but were abandoned in favour of the constraint-based notation, the reason being that more compact notations tend to become hard to read when the constraint is more complicated.

As a general rule, patterns on the right-hand side of a substitution rule get replaced even if their parent relation does not match the one on the left-hand side,

```
▷ q_{c d};
▷ @substitute!(%)( q_{a b} -> a Q_{a b} );
▷ c Q_{c d};
```

This follows the general logic of explicit wildcards,

```
▷ q_{c d};
▷ @substitute!(%)( q_{a? b?} -> a? Q_{a? b?} );
▷ c Q_{c d};
```

If this is not what you want, just rename the dummy index labels,

```
▷ q_{c d};
▷ @substitute!(%)( q_{e f} -> a Q_{e f} );
▷ a Q_{c d};
```

4.17 Procedures

Although the present version of cadabra does not contain a full-fledged programming language, it does allow you to construct groups of commands which can be applied term-by-term on a large sum. This is sometimes useful if e.g. the application of a command on a single term already leads to a large intermediate result.

Procedures are defined by making use of the `@procedure` construction. Here is an example, which distributes all products and performs a substitution:

```
▷ @procedure{ExpandAndCollect};
▷ @distribute!(%);
▷ @substitute!(%)( A C -> Q );
▷ @collect_terms!(%);
▷ @procedure_end;
```

A sample session which calls this procedure is

```
▷ (A+C)*(B+C+A) + C*(E+A);
▷ @call{ExpandAndCollect};
```

This produces some status output, followed by

```
A B + 2 Q + A A + C B + C C + C E + Q;
```

Note how the commands inside the procedure only act on each term in turn. The `@collect_terms` command has only collected two `Q` symbols which arose from the first term. Note also that all output of the commands inside the procedure has been suppressed, despite the appearance of the semi-colon line delimiter.

4.18 Reserved node names

There is a small number of node names which is reserved to always mean the same thing, i.e. for which the properties are not determined by the property system. These can be obtained using the '@reserved' command. Explicitly, they are

\anticommutator, \arrow, \comma, \commutator, \conditional, \frac, \cdot, \equals, \expression, \factorial, \indexbracket, \infty, \label, \pow, \prod, \regex, \sequence, \sum, \unequals.

Moreover, cadabra uses a number of standard names for mathematical operators as reserved keywords, to wit

\sin, \cos, \tan, \sinh, \cosh, \tanh, \arcsin, \arccos, \arctan, \sqrt

Some properties of these nodes are still obtained from the property system in order to make algorithms look more uniform, but the names of these reserved nodes can not be changed.

\anticommutator
Denotes the anti-commutator of two objects.

The default properties of \anticommutator are

```
\anticommutator{#}::IndexInherit.
\anticommutator{#}::Derivative.
```

\arrow
Denotes a substitution rule.

\comma
Denotes comma-separated objects. The parser rewrites any set of objects which are separated by an infix comma using the \comma object.

\commutator
Denotes the commutator of two objects.

The default properties of \commutator are

```
\commutator{#}::IndexInherit.
\commutator{#}::Derivative.
```

\conditional
Denotes a conditional pattern. The parser rewrites an infix vertical bar operator using a \conditional object, in which the first argument is the pattern and the second argument is the condition under which the pattern holds.

\frac
Denotes a generic ratio of objects.

\cdot
Denotes a dot product of vectors in which the contracted indices are suppressed. Displays as an infix dot.

`\equals`
Denotes equalities. The parser rewrites an infix = operator using an `\equals` object.

`\expression`
Denotes the top-level node of an expression.

`\factorial`
Denotes the factorial of an object. The parser rewrites a post-fix exclamation mark as a `\factorial` object.

`\indexbracket`
Denotes a group of objects with indices which are collectively written, as in $(A + B + C)_{mn}$.

The default properties of `\indexbracket` are

```
\indexbracket{#}::Distributable.
\indexbracket{#}::IndexInherit.
```

`\infty`
Denotes infinity.

`\label`
Denotes the label of an expression.

`\pow`
Denotes a generic exponent of one object raised to the power of another object. The parser translates the infix "`**`" to `\pow`.

The default properties of `\prod` are

```
\pow{#}::DependsInherit.
```

`\prod`
Denotes a generic product of objects. The parser translates objects separated by whitespace or by a "`*`" symbol to products.

The default properties of `\prod` are

```
\prod{#}::Distributable.
\prod{#}::IndexInherit.
\prod{#}::CommutingAsProduct.
\prod{#}::DependsInherit.
\prod{#}::WeightInherit(label=all, type=Multiplicative).
\prod{#}::NumericalFlat.
```

`\regex`
Denotes a regular expression name pattern. The first argument is a symbol and the second argument is a string containing the regular expression.

`\sequence`
Denotes a sequence or range of objects. The parser rewrites an infix `..` operator (two dots) using a `\sequence` object.

`\sum`
Denotes a generic sum of objects. The parser translates objects separated by a "+" symbol tol sums.

The default properties of `\sum` are

```
\sum{#}::CommutingAsSum.
\sum{#}::DependsInherit.
\sum{#}::IndexInherit.
\sum{#}::WeightInherit(label=all, type=Additive).
```

`\unequals`
Denotes inequalities. The parser rewrites an infix `!=` operator using an `\unequals` object.

4.19 Startup and program options

There are a few startup options for the command line version of cadabra. If present, a startup file `~/.cadabra` will be read when the program is started (only when running cadabra in command-line mode). This file can contain any cadabra input.

`--bare`
Disable reading of the startup file `~/.cadabra`.

`--silent`
Disable normal output (so that the only remaining output is through output redirection into files).

`--input [filename]`
Before switching to interactive mode, first read and process the indicated file.

`--prompt [string]`
Set the cadabra prompt to the given string.

`--silentfail`
Silently fail (i.e. do not complain) if an algorithm is activated which cannot be applied.

`--loginput`
Write the input into the debug file as well.

`--nowarnings`
Suppress displaying of any warnings which are not fatal errors.

`--texmacs`
Send output in TeXmacs format.

In order to have command-line editing functionality, the `prompt` program is provided. So the two ways of starting cadabra are

```
cadabra
prompt cadabra
```

the second of which gives cursor control and an editing history. There are various other settings of cadabra which can be influenced through environment variables; see section **??** for details.

4.20 Environment variables

There are a few environment variables which may be set to influence the behaviour of cadabra:

1. **CDB_LOG** When set, debug logging will be enabled.
2. **CDB_ERRORS_ARE_FATAL** If this variable is set, errors (inconsistency of the tree or attempted use of non-existing algorithm) will abort the program (the default action, when this variable is unset, is to ignore the offending input line).
3. **CDB_PARANOID** Perform extensive consistency checks on the internal data format after each algorithm has been applied (only useful for debugging).
4. **CDB_PRINTSTAR** Determines whether or not multiplication star symbols are displayed.
5. **CDB_TIGHTSTAR** Make output of multiplication symbols more compact.
6. **CDB_TIGHTBRACKETS** Make output of bracketed expressions more compact.
7. **CDB_USE_UTF8** Use UTF8 line breaking characters in the output.

Algorithm modules

All algorithms are stored in modules separate from the core of the program. These are at present all distributed together with the main program, but will in the future be available separately as binary or source plugin modules that can be loaded on demand.

Any expression that starts with a name beginning with an "@" symbol is interpreted as a command. Commands act on the currently selected objects in an expression or on the expression for which the label or number is given as the first argument. See section **??** for more details on how to apply algorithms to expressions.

5.1 Built-in core algorithms

There is a very small number of properties and algorithms built into the core program. These have to do with global memory management and global output settings.

properties:

algorithms:

5.2 Rationals, complex numbers and other numerics

properties:

algorithms:

5.3 Sums and products

This module handles expansion of sums and products as well as distributing products over sums and similar algorithms. It recognises

properties:

algorithms:

5.4 Young tableaux

Young diagrams (with unlabelled boxes) and Young tableaux (with labelled boxes) can be manipulated directly, and cadabra contains routines to e.g. compute tensor products or to put tableaux in standard form using all their symmetries. In order to avoid confusion about the meaning of "diagram" and "tableau", a Young tableau with labelled boxes is called a "FilledTableau" in cadabra, while a tableau without any labels in the boxes is a "Tableau".

properties:

algorithms:

5.5 Lists and ranges

When using list algorithms, be aware of the fact that when they act on an argument, they actually replace the argument, not generate a new one. Therefore, to generate a new expression which contains the length of another expression,

```
▷ A+B+C+D;
  1:= A+B+C+D;
▷ @length[@(1)];
  2:= 4;
```

This is to be compared with

```
▷ A+B+C+D;
  1:= A+B+C+D;
▷ @length(1);
  1:= 4;
```

in which case the first expression gets replaced by its length.

algorithms:

5.6 Properties

properties:

5.7 Dummy indices

Dummy indices are a subject by itself. There are various problems with most implementations of dummy indices in computer algebra software. In many cases, dummy indices are added as an afterthought, leading to situations were illegal expressions (e.g. with a more than twice repeated dummy index) can be entered. Even if this is flagged, most programs do not know about different index types (which leads to problems when there is more than one type of contraction, or a sum over only part of the index range). In cadabra, the system knows about dummy index *sets*, and all algorithms can request new dummy indices very easily. This is handled through the `dummies` module.

properties:

algorithms:

5.8 Symmetrisation and anti-symmetrisation

This module contains algorithms for the symmetrisation or anti-symmetrisation of expressions in their elements, and for the inverse procedure, whereby object trees are identified when they differ only by certain symmetrisations or anti-symmetrisations.

algorithms:

5.9 Field theory

properties:

algorithms:

5.10 Output routines

The core of cadabra only knows how to output its internal tree format in infix notation, and by default displays the result of every input line in this way (labelled by the expression number and label, if any). For more complicated output, for instance to feed cadabra expressions to other programs, or to see the internal tree form, routines are available in the `output` module. Note that these modules do not transform the tree or generate new expressions; they *only* generate output.

Related but *different* functionality is present in the `convert` module, which contains transformation algorithms which transform the internal tree to other conventions (e.g. in order to convert a cadabra tree to a tree which can be understood by Maple or Mathematica).

properties:

algorithms:

Furthermore, there are a few algorithms which are mainly useful for debugging purposes, as they display information about the internal representation of the expression tree. More information on the internal data structures can be found in section **??**.

algorithms:

5.11 General relativity

This module deals with special tensors relevant for general relativity. See in particular also the `canonicalise` algorithm in section **??**.

properties:

algorithms:

5.12 Substitution

Substitution of expressions into other ones is a subtle issue. One problem, namely that of relabelling of dummy indices, is addressed by the core routines and has been discussed in section **??**. Here we will discuss how cadabra handles the issue of pattern matching.

algorithms:

`@`
Given an expression number of label, this node replaces itself with a copy of this expression. As with any active node, this one takes care of dummy index relabelling automatically (see section **??**).

`@rename`
(documentation no longer correct!) Renames objects and numbered objects, but only on a fixed level, i.e. does not do pattern matching at all. For instance,

```
▷ F_{m1 m2 m3};
  F_{m1 m2 m3}
▷ @rename!(%){m}{r};
  F_{r1 r2 r3}
```

It will not prevent you from introducing dummy pairs:

```
▷ F_{m n};
▷ @rename!(%){m}{n};
  F_{n n};
```

However, it will relabel already existing dummy pairs if you rename an open index in such a way that it would conflict with the dummy pair:

```
▷ {m,n,p,q,r,s}::Indices(vector).
▷ F_{m n} G_{n q};
▷ @rename!(%){m}{n};
  F_{n p} G_{p q};
```

5.13 Linear algebra

algorithms:

5.14 Gamma matrix algebra and fermions

This module deals with manipulations of the Clifford algebra, spinor representations of the Lorentz group and related issues. Cadabra follows the conventions of Sevrin's appendix 1.5, except for the fact that we call the time-like component zero and the product of all gamma matrices $\tilde{\Gamma}$.

properties:

algorithms:

5.15 Conversion from/to other formats

Cadabra's syntax is a superset of the Maple and Mathematica formats, in the sense that the internal tree representation of cadabra can store input for both of these systems. However, this may not always be the most convenient way of storage, as cadabra often has more compact or expressive ways to store tensors. The `convert` module provides conversion algorithms from and to these two formats.

algorithms:

`@from_math`**[expression]**
Reads mathematica input format as used by GAMMA [**?**]. Fully anti-symmetric tensors are denoted with

```
Tensor[name,{indices}]
```

Tensors with property "weyltensor" are printed as

```
▷ W_{r1 r2 r3 r4}
Weyl[{r1,r2}, {r3,r4}]
```

(warning, this removes the tensor name).

`@to_math`**[expression]**
Converts the mathematica expression to a cadabra one. See above for details about the conversion process.

Core functionality

6.1 Source file description

The files in the `src` directory build up the core of the program and are described below. Algorithm-specific code is in the `src/modules` subdirectory and a description of those files can be found in section **??**. The files below should not be changed when adding new modules.

`main.cc`
: Startup routines; initialises the I/O streams, sets up the signal handlers for control-C and window resize signals and starts the main event loop.

`manipulator.cc`, `manipulator.hh`
: Contains the object that processes the input line by line, activates the parser on it, and scans the tree for active nodes. A few very low-level commands (see section **??**) are handled in this class, while the other ones are dispatched to external modules.

`preprocessor.cc`, `preprocessor.hh`
: The code for conversion of human editable infix input to the tree-form handled by the other parts of the program. This is a text-to-text filter forming the first pass of the parser. All functionality can be tested using `test_preprocessor.cc`.

`storage.cc`, `storage.hh`
: The node and tree classes for storage of the tree form of expressions. These are the classes on which actual symbolic manipulation takes place. See also the separate `tree.hh` class.

`parser.cc`, `parser.hh`
: The class which contains the parsing algorithms that turns output of the preprocessor class into a tree form.

`algorithm.cc`, `algorithm.hh`
: The base class for all the classes defined in the modules subdirectory. Plus the implementation of scanning for command arguments and the necessary logic to create the undo information, apply an algorithm until it no longer changes the expression and other things which are independent of the particular command.

`combinatorics.hh`
: General routines for the generation of combinations and permutations of elements in a vector. Consists of a single class `combinations<...>` templated over the type of the objects to be permuted. This class acts as a container holding both the original vector (in the `storage` member), the permutation patterns (in the `sublengths` vector, more on this later) as well as the resulting permuted vectors (accessible through `operator[]`; the total number of combinations is given by `size()`).

6.2 Tree representation of expressions

Expressions are internally stored in a tree container class, based on the `tree.hh` library [**?**]. The present section describes the tree format in some more detail.

Let us start with a couple of remarks on the motivation which has led to the current implementation. An important requirement for the storage format is that symbols should not have a fixed meaning, but rather be like variables in a computer program: they can be declared to represent objects of a certain type. Similarly, the user should be free to use arbitrary bracket types and sub- and superscript indicators (these should in addition be kept during the algebraic manipulations). The first issue is resolved with the property system, but the second one needs support directly in the storage tree.

Secondly, storage should be such that a trivial (often occurring) extensions from one object to another should not lead to blowup. For instance, the difference between `A_\mu` and `A_{\mu \nu}` should preferably not introduce a new layer between the `A` node and the `\mu` node. This is achieved by adding the bracket type to the *child* node rather than the parent (in the example above, the curly brackets are part of the `\mu` and `\nu` nodes). This applies also to e.g. sums: in `(A+B)` (which is converted by the preprocessor to `\sum(A)(B)`), the round brackets are part of the child nodes of the `\+` node.[7] Similarly, extending `q` to `3q` should not introduce new intermediate layers to group the two symbols together. This has been achieved by giving each node a multiplier field (this holds true even for products, i.e. `3(a+b)` is represented as a sum node with multiplier 3 (in addition, such numerical factors are always commuted to the front of the expression).[8]

It should also be possible to make expressions arbitrarily deeply nested; one should be able to write `a+b` as well as `\sin(a+b)`. This is in contrast to programs such as `Abra` and `FORM` which store their input as sums of products of factors, i.e. without further nesting.

Finally, tt should be posssible to store the history of an expression, in order to implement unlimited undo.

The data stored at each tree node is further explained in section **??**, see in particular the excerpt from the `storage.hh` file in figure **??**. The structure of the actual tree is illustrated in figure **??**. Each expression starts with a `history` node. The label of the expression is given in a subnode `label`. All remaining child nodes of the `history` nodes represent the history of the expression. The most often encountered node here is the `expression` node, which contains the actual expression. However, there are other nodes as well, for instance `asymimplicit` which contains information about the symmetry structure of an expression (this feature is not yet enabled).

6.3 Nested sums, products and pure numbers

Nested sum and product nodes require some explanation, as the way in which cadabra handles and stores these determines how brackets can appear in sums and products. As a general rule, sums within sums which are such that all child nodes have the same, non-visible bracket are not allowed, and automatically converted to a single sum. In other words

```
(\sum{a}{b}+\sum{c}{d})
```

[7] Subsequent child nodes with the same bracket type associated to them are taken to be part of the same group; the input `A_{\mu}_{\nu}` will therefore be printed as `A_{\mu \nu}`. Similarly, there is no way to represent $(a)+(b)$ since the internal storage format automatically turns this into $(a+b)$.

[8] The multiplier field is a rational, and is intended to stay that way. It is not a good idea to make it complex, otherwise we will end up with octonionic multipliers at some stage.

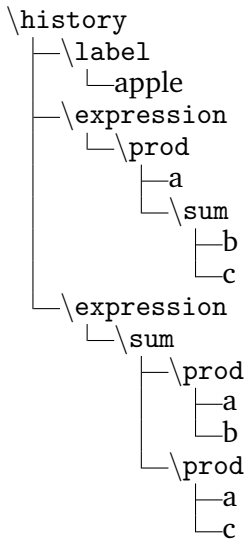

Figure 1: Example history tree for a single expression. Here, the program started with the user entering the expression $a(b+c)$. This expression was then manipulated by applying the `@distribute` algorithm. The nodes at the top of the expression tree are all of the `history` type.

is not allowed, since it would print as `(a+b+c+d)` which is indistinguishable from

```
(\sum{a}{b}{c}{d}) .
```

The first expression is therefore internally always converted to the second one before it is stored. A similar situation occurs with products, where

```
\prod{a}{\prod{b}{c}}
```

is not allowed and "flattened" to the simpler

```
\prod{a}{b}{c} .
```

The general rule is that nested sums and products are converted to flattened form if these two forms are indistinguishable in the standard output format. Nested sums and products can occur, but they have to have different or non-empty bracket types for their children. An example is

```
\prod{a}{\prod(b)(c)}
```

which prints as `a*(b*c)`. This is not automatically converted to `a*b*c`. Note that this form is *not* represented as

```
\prod{a}(b)(c)    (wrong)
```

This is a consequence of the second rule: all child nodes of product or sum nodes must have the same bracket type. In summary, automatic flattening of nested sum or product trees do not automatically occur over bracket boundaries. You can then write and keep $(\sin(x)+\cos(x))+(\sin(x)+3\cos(x))$, which will be stored as

```
\sum{\sum(sin(x))(cos(x))}{\sum(sin(x))(cos(x))}
```

In order to remove brackets, one uses the `@sumflatten` and `@prodflatten` commands, see section **??**.

Products and sums with sub- or superscripts which apply to one or more of the child nodes are internally represented using `\indexbracket` nodes. That is, the input

```
q (a+b)_\mu
```

is represented as

```
\prod{q}{\indexbracket{\sum(a)(b)}_{\mu}}
```

(note the way in which the bracket types are inherited). In fact, such indexbracket constructions apply for any content inside the brackets; in the example above, an indexbracket would be generated even without the `\sum` node: the input

```
q (\Gamma^r)_{a b}
```

is stored as

```
\prod{q}{\indexbracket(\Gamma^r)_{a b}}
```

The result of the indexbracket way of storing expressions is that "index free" manipulations apply directly inside the indexbracket argument and no special algorithms are needed. Input will automatically be converted to this form. Getting rid of indexbrackets around single symbols is done using `@remove_indexbracket`.

Another issue is the storage of numbers. Since pure numerical factors are only allowed to appear in the multiplier field of a node, a pure number is stored as "1" with its multiplier equal to the number. Product nodes with two arguments of which one is a number are not allowed and will be converted to a single node with multiplier field, automatically.

In order to eliminate ambiguities concerning multipliers in sum and product nodes, the following rules should be obeyed: sum nodes always have unit multiplier (i.e. nontrivial multipliers are associated to the children) and children of product nodes should also always have unit multiplier (i.e. the multipliers of all the children should in this case be collected in the prod node itself).

6.4 External scalar engine interface

Cadabra is a tensor computer algebra system, without much functionality for simplification or manipulation of scalar expressions. Other systems exist which handle this much better, and it is a waste to duplicate those efforts. Therefore, the `algorithm` class contains functionality to isolate scalar factors from a tensorial expression, manipulate them using an external scalar CAS, and re-insert them into the internal cadabra tree.

Scalar expressions are defined to be sums or (subsets of factors of) products which contain no objects with subscript or superscript children (i.e. contain no tensors).

Exporting a scalar expression to an external CAS requires a rewriting step before and after the export. At present only Maxima's format is supported.

Module implementation guide

7.1 Storage of numbers and strings

There is a special memory area for the storage of rational numbers (with arbitrary-length integers for the numerator and denominator) as well as for the storage of strings.

7.2 Accessing indices

Indices are stored as ordinary child nodes. However, there are various ways to iterate over all indices in a factor, which are more useful than a simple iteration over all child nodes.

7.3 Object properties

The properties attached to a node in the tree can be queried by using a lookup method in the `properties` namespace. Object properties are child classes of the `property` class, and contain the property information in a parsed form. The following code snippet illustrates how properties are dealt with:

```
AntiSymmetry *s=properties::get<AntiSymmetry>(it)
if(s) {
   // do something with the property info
   }
```

Here we used a property called `AntiSymmetry`, which is declared in the the module header `modules/algebra.hh` (see the other header files of the various modules for information about the available property types). If the node pointed to by `it` has this property associated to it, the pointer `s` will be set to point to the associated property node. **[KP: Better to use a property example where the property node contains some additional information, which can then be used.]**

It is also possible to do this search in the other direction, that is, to find an object with a given property.

```
properties::property_map_t::iterator
               it=properties::get_pattern<GammaMatrix>();
if(it!=properties::props.end()) {
   // Now *(it->first) contains the name of a GammaMatrix object.
   }
```

[KP: Instead of checking these things, it is better to throw exceptions, since that will make the code more readable and also the error handling more uniform (it can go in one place instead of being repeated).]

Sums and products inherit, in a certain way, properties of the terms and factors from which they are built up. This is true also for other functions. If you want to know whether a given node has such *inherited properties*, a simple call to `get<...>` is not sufficient. Instead, use the following construction:

```
Matrix *m=properties::get_composite<Matrix>(it)
if(m) {
   // do something with the property info
   }
```

These inheritance rules for composite objects are at present hard-coded but will be user-defineable at a later stage.

In addition to querying for object properties, some algorithms also act on objects which encode their property in a special reserved node name. As an example, the `@prodrule` acts on objects which carry a `Derivative` property, but it also acts on `@cdb_Derivative` objects. This allows e.g. the `@vary` algorithm to wrap a derivative-like object around powers and have `@prodrule` expand that

7.4 Writing a new algorithm module

All functionality of cadabra that goes beyond storage of the expression trees is located in separate modules, stored in the `src/modules` subdirectory. New functionality can easily be added by writing new modules. All algorithm objects derive from the `algorithm` object defined in `src/algorithm.hh`, and only a couple of virtual members have to be implemented.

The class interface for the algorithm object is shown in figure **??**. The members which any class derived from `algorithm` should implement are described below.

`constructor(exptree&, iterator)`
: Algorithm objects get initialised with a reference `tr` to the expression tree on which they act, together with an iterator `this_command` pointing to the associated command node. By looking at the child nodes of the latter, the arguments of the command can be obtained. The simplest way to do this is to use the sibling iterators returned by `args_begin()` and `args_end()`; these automatically skip the argument which refers to the equation number (i.e. they skip the `(5)` argument in `@command(5){arg1}{arg2}`). The constructor is a good place to parse the command arguments, since that will in general only be required once. Do not do anything in the constructor that has to be done on every invocation of the same command on different expressions.

  If you discover that the arguments passed to an algorithm are not legal, you have to throw an `algorithm::constructor_error` exception.

`description()`
: Should display an explanation of the functionality of the algorithm on the `txtout` stream. Do not add `std::endl` newlines by hand, these will be inserted automatically.

`bool can_apply(iterator)`
: Determines whether the algorithm has a chance of acting on the given expression. This is an *estimate* but is required to return `true` if there is a possibility that the algorithm would yield a change. It is allowed to return `true` and then still not do anything when `apply` is called.

```
class str_node {
   public:
      ...
};
```

Figure 2: The node class.

```
class algorithm {
  public:
    typedef exptree::iterator         iterator;
    typedef exptree::sibling_iterator sibling_iterator;

    algorithm(exptree&, iterator&);

    virtual void     description() const=0;
    virtual bool     can_apply(iterator&);
    virtual bool     can_apply(sibling_iterator&, sibling_iterator&);
    virtual result_t apply(iterator&);
    virtual result_t apply(sibling_iterator&, sibling_iterator&);

    sibling_iterator args_begin() const;
    sibling_iterator args_end() const;
    unsigned int     number_of_args() const;

    exptree&  tr;
    iterator this_command;
};
```

Figure 3: The relevant members of the class interface for the `algorithm` object. The virtual members and the constructor have to be implemented in new algorithm objects; see the main text for an explanation of the two versions of `can_apply` and `apply`.

```
class exptree {
  public:
    sibling_iterator arg(iterator, unsigned int) const;
    unsigned int     arg_size(sibling_iterator) const;
    multiplier_t     arg_to_num(sibling_iterator, unsigned int) const;
};
```

Figure 4: The relevant members of the class interface for the `exptree` object.

It takes an `iterator` as argument, which indicates the node at which the algorithm is supposed to act.

Since `can_apply` is guaranteed to be called before `apply`, it is allowed for the `can_apply` member to store results for later use in `apply`. However, multiple calls can be made to this combo, so any data stored should be erased upon each call to `can_apply`.

`apply_result apply(iterator&)`

This is where the actual algorithm is applied. It is allowed to modify the iterator if necessary. No elements in the tree which sit above the entry point should ever be changed.

It should also be disallowed that an algorithm unwraps the single node which it has been given into multiple nodes, because the returned iterator cannot propagate this information to the caller. `cleanup_nests` does this wrong. **FIXME**

Before exiting this function, the member variable `expression_modified` has to be set according to whether or not the apply call made any changes to the expression.

If the algorithm can take a long time to complete and you want to give the user the option of interrupting it with control-C, you can check (at points where the algorithm can be interrupted) the global variable `interrupted`. If it is set, you should immediately throw an exception of type `algorithm_interrupted`, preferably with a short string describing the location or algorithm at which the interrupt occurred. The exception will be handled at the top level in the `manipulator.cc` code, where the current expression will be removed.

Algorithms can write progress information to the stream `txtout` and debug information (which will go into a separate file on disk) to `debugout`. These are global objects. Do not use the C++ streams `std::cout` and `std::cerr` for this purpose.

An algorithm is *never* allowed to modify the tree above the node or range of sibling nodes which were passed to the `apply` call. In other words, siblings and parent nodes should never be touched, and the nodes should not be removed from the tree directly. If you want to remove nodes, this can be done by setting their multiplier field to zero; the core routines will take care of actually removing the nodes from the tree. It is allowed to inspect the tree above the given nodes, but this is discouraged. What does this mean? **FIXME**

Upon any exit from an algorithm module, the tree is required to be in a valid state. In many cases, it is useful to clean up modifications to the tree by calling the utility function `cleanup_expression` (see section **??** for more information).

7.5 Adding the module to the system

Once you have written the `.hh` and `.cc` files associated to your new algorithm, it has to be added to the build process and the core program. First, add the header to the master header file `src/modules/modules.hh` file; this one looks like

---

`src/modules/modules.hh:`

```
#include "algebra.hh"
#include "pertstring.hh"
```

```
#include "select.hh"
...
```

You also have to add it to the `src/Makefile.in`. Near the top of this file you find a line looking somewhat like

```
src/Makefile.in:

...
MOBJS=modules/algebra.o modules/pertstring.o modules/convert.o \
      modules/field_theory.o modules/select.o modules/dummies.o \
      modules/properties.o modules/relativity.o modules/substitute.o
...
```

and this is where your module has to be added. Finally, you have to make the algorithm visible to the core program. This is done in the `src/manipulator.cc` file. In the constructor of the `manipulator` class (defined somewhere near the top), you see a map of names to algorithm classes. A small piece is listed below:

```
src/manipulator.cc:

...
// pertstring
algorithms["@aticksen"]       =&create<aticksen>;
algorithms["@riemannid"]      =&create<riemannid>;

// algebra
algorithms["@distribute"]     =&create<distribute>;
 ...
```

Add your algorithm here; it does not have to have the same name as the object class name. Once this is all done, rerun configure and make.

7.6 Adding new properties

[KP: Discuss the various member functions of property, in particular the way in which parse is supposed to behave, and the way in which properties should be registered.] Since algorithms rely crucially on a *fixed* set of properties which they provide themselves, we store these things in a C++ way, rather than using strings. Another motivation: we do not want to parse (and check) these property trees every time they are accessed.

Properties typically carry key/value pairs which further specify the characteristics of the property. When the property object is created, the core calls the `parse()` method of the property, in which you are responsible for scanning through the list of key/value pairs. This is rather simple: just call the `find()` member of the `keyval` argument,

```
keyval_t::iterator ki=keyvals.find("dimension");
if(ki!=keyvals.end()) {
```

```
class property\_base{
  public:
    virtual bool        parse(exptree&, iterator pat, iterator prop);
    virtual std::string name() const=0;
    virtual void        display(std::ostream&) const;
};

class property      : public property\_base {};
class list\_property : public property\_base {};

class labelled\_property : public property {
  public:
    std::string label;
};
```

Figure 5: The relevant members of the class interface for the `property_base` object and the derived objects.

```
    // now ki->second is an iterator to an expression subtree
    }
```

You are supposed to remove any handled keyval pair from the list by using the `erase()` member, so that the system can keep track of unhandled pairs. Do not forget to call the `parse()` method of all parent classes at the end.

Once this is all done, register the property with the system by adding an appropriate call to the module's `register_properties()` method, found at the top of each module.

7.7 Throwing errors

If, at any stage, an inconsistent expression is encountered, the safe way to return to the top level of the manipulator is to throw the exception class `consistency_error` (defined in `algorithm.hh`).

7.8 Manipulating the expression tree

The `exptree` class contains a number of tree access routines which enhance the functionality of the `tree.hh` library.

`index_iterator`

`index_map_t`

Acting on products or single terms

use `is_single_term` inside `can_apply` to test for this case. Then in `apply` call `prod_wrap_single_term` (which will be harmless for proper products). At the very end of `apply`, call `prod_unwrap_single_term` to remove the product again.

7.9 Utility functions

There are also various useful helper functions in the `exptree` class, mainly dealing with the conversion of expression numbers or labels to iterators that point to the actual expression in the tree.

`cleanup_expression(exptree&)`
`cleanup_expression(exptree&, iterator)`
: Converts an expression (or part of it below the indicated node) to a valid form. That is, it calls various simplification algorithms on the expression, among which `sumflatten` and `prodcollectnum`.

`cleanup_nests`
: When called on a product inside a product, or a sum inside a sum, and the bracket types of both node and parent are the same, flatten the tree. Adjusts the iterator to point to the product or sum node which remains.

`equation_by_number_or_name`
: Given an iterator to a node that represents an expression number or expression label, this member returns an iterator that points to the actual node (to be precise: to the `\history` node of the expression).

`arg_size(sibling_iterator)`
`arg(iterator, unsigned int)`
: Many algorithms require one *or more* objects as arguments. These are usually passed as lists, and therefore end up being of two forms: either they are single objects or they are `comma` objects representing a list. In order to avoid having to check for these two cases, one can use these two member functions.

  The iterator arguments of these two member functions should point to the child which is either a single argument or a `\comma` node.

`pushup_multiplier`
: Can be called on any node. If the node has a non-unit multiplier and the storage conventions exclude this, the algorithm will push the multiplier up the tree.

7.10 Writing a new output module

[KP: Structure is different now: one inherits from `output_exptree` and then has printing stuff at ones disposal. Still would be nice to be able to change the printing objects for a given node type, i.e. modify the map of node names to printing objects.]

[KP: Also do something intelligent with the Mathematica output flags].

7.11 Namespaces and global objects

The following objects are global:

`modglue::opipe txtout`
: The normal output stream, to which you should communicate with the user.

`std::ofstream debugout`
The debug output stream, which normally writes to a file, and should only be used to write output which is useful for debugging purposes.

`nset_t name_set`
The global collection of all strings. Nodes in the expression tree contain an iterator into this map (see in particular figure **??**).

`rset_t rat_set`
The global collection of all rational numbers. Nodes in the expression tree contain an iterator into this map (see in particular figure **??**).

`bool interrupted`
A global flag which indicates that the user has requested the computation to be interrupted. See section **??** for more information on this variable.

`stopwatch globaltime`
The wall time since program start.

`unsigned int size_x, size_y`
The size of the current output window.